

MÁSTER UNIVERSITARIO EN TECNOLOGÍAS WEB,
COMPUTACIÓN EN LA NUBE Y
APLICACIONES MÓVILES



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE MÁSTER

MEDICIÓN DE CONSUMO ENERGÉTICO DE
CARGAS TRABAJO DE CODIFICACIÓN DE VIDEO
SOBRE CONTENEDORES

AUTOR: GUILLERMO BLÁZQUEZ ORTEGA

TUTORA: JUAN GUTIERREZ AGUADO

JULIO 2026

Declaración de autoría:

Yo, Guillermo Blázquez Ortega, declaro la autoría del Trabajo Fin de Máster titulado “Medición de Consumo Energético de cargas trabajo de codificación de video sobre contenedores” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Firmado:

Resumen:

Conocer el consumo energético que una carga de trabajo necesita es un aspecto esencial en el contexto actual, en el cual la mayoría de la carga de computación se lleva a cabo en grandes CPDs empleando arquitectura de contenedores. Una toma de métricas constante permite realizar estimaciones precisas, aportando mucha información relevante a la hora de definir diferentes aspectos técnicos.

El siguiente Trabajo Fin de Máster gira en torno a esta premisa y a las problemáticas que ello implica, estando enfocado en la codificación de video de alta resolución sobre contenedores que emplean CPU y GPU física para acelerar el proceso. El objetivo es lograr la mejor configuración de codificación de video para FFmpeg que consuma la menor cantidad de energía sin empobrecer la calidad. Para poder lograrlo, se ha requerido una herramienta que permita medir el consumo de procesos individuales, tanto sobre la CPU como en la GPU, y para ello se ha realizado una comparativa entre herramientas, las cuales están pensadas para medir el consumo de trabajo de tareas desplegadas sobre contenedores.

Una vez se ha seleccionado la herramienta indicada, se han realizado un conjunto de baterías de pruebas que me han permitido llegar a las CONCLUSIONES.

Abstract:

This is the abstract of the TFM. It must be short and cover the main aspects of the TFM. esto es una prueba de escritura usando el

Resum:

Aquest és el resum del TFM. Ha de ser curt (màxim mitja pàgina) i cobrir els aspectes principals del TFM.

Agradecimientos:

Me gustaria agrader el apollo de mi tutor Juan Gutierrez Aguado, quien me ha acompañado en todo el proceso de elaboracion y redaccion del siguiente trabajo fin de master. Gracias a Juan, he podido emplear referencias de valor y definir pruebas que demuestren la relevancia de mi trabajo.

Índice general

1. Introducción	9
1.1. Contexto del Trabajo	9
1.2. Motivaciones y Objetivos	11
2. Estado del Arte	13
2.1. Contexto Global Consumo Energetico	13
2.1.1. Generacion Energetica	14
2.1.2. Eficiencia Energetica	14
2.2. Especificaciones Tecnicas del Entorno de Pruebas	14
2.3. Selecccion de Herramientas de Consumo Energetico	15
3. Desarrollo de Proyecto	19
3.1. Configuración de Entorno & Herramientas	19
3.1.1. Selecccion e Intalacion del Sistema Operativo	19
3.1.2. Configuración e Instalación de Herramientas	21
3.1.3. Estado Final del Sistema	25
3.2. Selección de Herramienta para Métricas de Consumo	26
3.2.1. Muestra de Videos Para la fase de Experimentacion 1	26
3.2.2. Metricas de consumo: Transcodificacion usando 1 contendor sobre CPU+GPU	26
3.2.3. Metricas de consumo: Transcodificacion usando 1 contendor sobre CPU	26
3.2.4. Metricas de consumo: Transcodificacion usando 1 contenedor con 3 cores de CPU limitados	26
3.2.5. Metricas de consumo: Transcodificacion usando 2 contenedores simultaneos con limite de 3 cores cada uno	26
3.2.6. Conclusiones fase 1 de la experimentacion	26
3.3. Fase de Experimentacion 2: Consumo energetico en relacion al SITI y el Numero de Threas	26
3.3.1. Muestra de videos para la fase 2 de experimentacion	26
3.3.2. Selecccion de muestras Basado en SI y TI	26

3.3.3. Consumo energetico en relacion al SITI	27
3.3.4. Consumo energetico al limitar threads y numero de cores	27
4. Pruebas y Resultados	29
5. Conclusiones y Trabajo Futuro	31
A. Apéndice	33
A.1. Subapendice: Experimentacion	33
A.1.1. Experimetacion Fase 2	33
Bibliografía	33

Capítulo 1

Introducción

1.1. Contexto del Trabajo

En la actualidad, la carga computacional ha migrado del dispositivo local hacia grandes Centros de Procesamiento de Datos (CPD), los cuales emplean una arquitectura de microservicios basada en contenedores y gestionada por orquestadores como Kubernetes. Este cambio de paradigma ha permitido aumentar la capacidad de los servicios y aplicaciones para dar servicios a múltiples nuevos clientes, así como la ejecución simultánea de tareas de alta computación en paralelo, las cuales antes estaban limitadas por la capacidad hardware del proveedor. Como consecuencia, el consumo de recursos se ha visto altamente afectado, siendo uno de los más afectados el consumo energético, el cual, según se indica en [1], supone entre el 15 y el 20 % del consumo global energético.

Siendo más precisos, basándome en el artículo [2], sabemos que la navegación por internet supone un 3.7 % de los gases de efecto invernadero, del cual el 65 % corresponde al streaming de video. Esto supone que el streaming de video es responsable de las emisiones de efecto invernadero equivalentes a las liberadas por la industria de transporte aéreo, como se cita en el paper [2].

Es por este motivo que la obtención de métricas precisas, segmentadas por proceso y por área de hardware utilizada, se ha convertido en una necesidad vital. Tanto en el ámbito de la estimación de usos y costes como en la gestión de análisis de sostenibilidad, siendo estos últimos esenciales para la toma de decisiones a futuro, tanto en el contexto industrial como en el contexto socialpolítico.

Bajo este contexto, se desarrolla el siguiente Trabajo de Fin de Máster (TFM), siguiendo una de las líneas de investigación del proyecto Energy-Conscious Optimization for Cloud-Edge Video Ecosystems (ECO-STREAM). Este proyecto tiene como objetivo la optimización de cargas de trabajo de codificación y streaming de vídeo de alta calidad, en arquitectura de contenedores, mediante el uso de CPU y GPU físicas.

El objetivo principal de este trabajo de fin de máster es definir la configuración de codificación de alta resolución que minimice el consumo energético sin comprometer la calidad del resultado final. Para ello, es indispensable contar con herramientas capaces de medir el consumo energético a nivel de contenedor de forma precisa, segmentando el gasto entre CPU y GPU.

Para la adquisición de métricas, se propone el uso de herramientas software adaptadas para el contexto de las cargas de trabajo, utilizando la medición de consumo energético

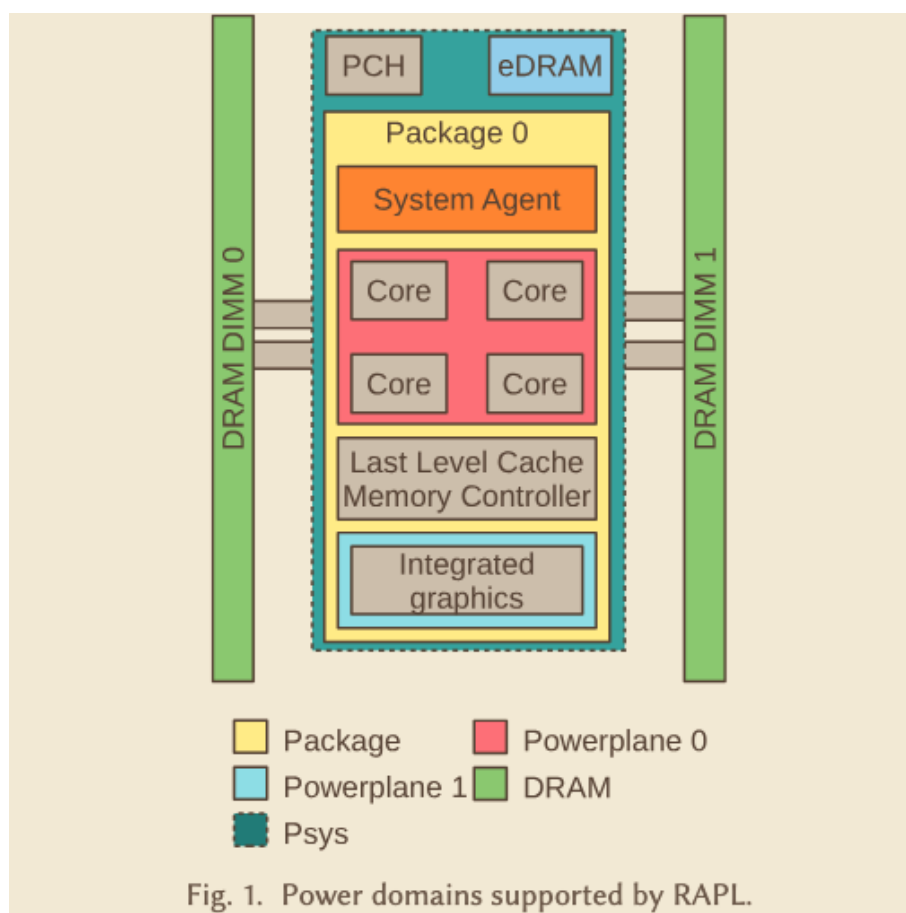


Figura 1.1: Agrupación de componentes en metrica de RAPL

total del sistema como punto de comparación y verificación. El uso de herramientas hardware se ha descartado debido a su complejidad de implementación, así como a los desafíos de mantenimiento y escalabilidad que presentan en entornos de producción.

Las herramientas seleccionadas utilizarán Running Average Power Limit Energy Reporting (RAPL) [3], una aplicación de Intel que permite extraer métricas precisas del consumo energético de varios componentes del sistema, junto con la API interna de NVIDIA y sus librerías asociadas. Cabe destacar que RAPL agrupa en grupos y subgrupos los componentes del sistema, como se observa en la imagen 1.1, permitiendo obtener métricas precisas.

Una vez definido el contexto y la problemática que se desea solventar en la introducción 1, así como los objetivos y motivaciones del proyecto. Posteriormente, en el capítulo 2 (Estado del Arte) se definirá el marco teórico y contexto tecnológico, el cual será la base con la que justificaremos las decisiones tomadas a lo largo de este trabajo. Seguidamente, en el capítulo 3 (Desarrollo del Proyecto) se describirán los diferentes pasos, configuraciones y problemáticas encontradas durante el desarrollo del trabajo. En el capítulo 4 (Resultados y Pruebas), se hará uso de los datos y resultados obtenidos del apartado anterior y se agregarán diferentes tablas y ayudas gráficas, las cuales facilitarán la comprensión y el desarrollo de las conclusiones. Para finalizar, en el capítulo 5 (Conclusiones y Trabajo Futuro), se expondrán las conclusiones y reflexiones acerca del trabajo realizado. Destacar que al final de este trabajo también se podrá encontrar la bibliografía A.1.1 y un anexo A, el cual contendrá información adicional.

1.2. Motivaciones y Objetivos

El presente trabajo se centra en la identificación de configuraciones de codificación de vídeo energéticamente eficientes mediante el uso de FFmpeg sobre entornos basados en contenedores. La gran cantidad de parámetros disponibles durante el proceso de codificación dificulta la selección de aquellas configuraciones capaces de proporcionar una elevada calidad visual manteniendo un consumo energético reducido. Esta problemática adquiere una especial relevancia en entornos cloud y de computación distribuida, donde pequeñas mejoras en la eficiencia pueden traducirse en reducciones significativas del consumo energético y de los costes operativos.

Para abordar este problema, resulta necesario disponer de mecanismos que permitan medir de forma precisa el consumo energético asociado a cada carga de trabajo. Sin embargo, las herramientas existentes presentan limitaciones en términos de precisión, granularidad o capacidad de integración con entornos basados en contenedores. Por este motivo, una parte fundamental del trabajo consiste en el análisis de las soluciones disponibles y en el desarrollo de una herramienta propia capaz de monitorizar con precisión el consumo energético de los recursos hardware involucrados, especialmente la CPU y la GPU.

La motivación principal de este trabajo se encuentra alineada con los objetivos del proyecto ECO-STREAM, cuyo propósito es mejorar la eficiencia energética de los procesos de codificación y transmisión de vídeo. La optimización de estos procesos permite un mejor aprovechamiento de los recursos computacionales, una reducción del consumo energético y, en consecuencia, una disminución de la huella de carbono asociada a las infraestructuras de procesamiento multimedia.

Asimismo, este trabajo pretende aportar una metodología reproducible para la evaluación energética de cargas de trabajo de codificación de vídeo, así como información de utilidad para futuras investigaciones relacionadas con la monitorización energética, la optimización de aplicaciones multimedia y el desarrollo de infraestructuras cloud más sostenibles.

Capítulo 2

Estado del Arte

2.1. Contexto Global Consumo Energetico

Bajo el paradigma actual de la computación distribuida pueden distinguirse dos modelos principales de despliegue en la nube [4]: las nubes públicas y las nubes privadas. Las primeras ofrecen recursos de computación como un servicio accesible para terceros, permitiendo a los clientes utilizar la capacidad de procesamiento de grandes Centros de Procesamiento de Datos (CPDs) sin necesidad de gestionar ni mantener la infraestructura subyacente. Por el contrario, las nubes privadas son administradas por la propia organización sobre una infraestructura dedicada, lo que implica asumir costes adicionales asociados al mantenimiento del hardware, la administración de los sistemas y el consumo energético necesario para garantizar su funcionamiento.

La creciente adopción de estos modelos de computación ha impulsado la construcción de grandes infraestructuras capaces de atender millones de peticiones de forma simultánea. Entre ellas destacan las Redes de Distribución de Contenido (CDN, Content Delivery Networks), cuya finalidad es distribuir contenido multimedia a través de redes de alta velocidad para reducir la latencia y mejorar la experiencia de los usuarios. Tanto los CPDs como las CDN requieren una gran cantidad de recursos energéticos para mantener un funcionamiento continuo las 24 horas del día, los 365 días del año [5]. Este consumo no se limita únicamente a los equipos de procesamiento, sino que también incluye los sistemas de refrigeración necesarios para mantener unas condiciones de operación adecuadas, siendo el agua uno de los recursos más utilizados en la industria para este fin

El incremento del número de CPDs y de la capacidad de procesamiento requerida ha supuesto un importante impacto desde el punto de vista medioambiental. Según el informe publicado por la International Energy Agency (IEA) [6], la demanda energética asociada a los centros de datos continúa aumentando debido al crecimiento de las cargas de trabajo digitales, lo que ha impulsado la necesidad de ampliar la capacidad de generación eléctrica [7]. Entre los principales factores responsables de este incremento destacan el auge de la inteligencia artificial, el procesamiento masivo de datos y la construcción de nuevas infraestructuras tecnológicas, como centros de datos y redes de distribución de contenido, que requieren un suministro energético continuo y sistemas de refrigeración de gran capacidad.

Como consecuencia de esta elevada demanda, numerosos proyectos tecnológicos se enfrentan actualmente a limitaciones en el acceso a la red eléctrica. El informe de la IEA indica que existen solicitudes de conexión que representan aproximadamente 2.500 GW

de nueva demanda eléctrica, muchas de las cuales permanecen en espera debido a la insuficiente capacidad de generación y distribución energética. Aunque el propio informe señala que parte de estas solicitudes corresponden a proyectos que finalmente podrían no materializarse, pone de manifiesto que el crecimiento de la demanda energética constituye un desafío real y de alcance global.

2.1.1. Generacion Energetica

Como respuesta al incremento de la demanda energética y a la necesidad de reducir el impacto medioambiental, durante los últimos años se han producido cambios significativos tanto en las fuentes de generación eléctrica como en su distribución. Según el informe publicado por la International Energy Agency (IEA) [7], la tendencia a medio y largo plazo apunta hacia un aumento progresivo de la participación de fuentes de energía renovables y de tecnologías de bajas emisiones. Entre ellas, la energía solar fotovoltaica destaca como la tecnología con mayor crecimiento a nivel mundial, impulsada principalmente por las inversiones realizadas por China y Estados Unidos durante los últimos años. Este crecimiento ha permitido reducir la dependencia del carbón en numerosos sistemas eléctricos, incluso en un contexto de aumento de la demanda energética. Asimismo, la biomasa continúa consolidándose como una alternativa renovable relevante debido al aprovechamiento energético de residuos, mientras que la energía nuclear ha alcanzado niveles históricos de generación eléctrica, contribuyendo a satisfacer el incremento de la demanda mundial.

No obstante, los combustibles fósiles continúan desempeñando un papel predominante en el sistema energético global. Aproximadamente el 35 % de la energía consumida procede del petróleo y alrededor del 20 % del gas natural, que continúa siendo una de las principales fuentes de generación eléctrica a nivel mundial. Aunque el incremento de las energías renovables y las mejoras en eficiencia energética han permitido limitar el crecimiento de las emisiones de dióxido de carbono, las emisiones globales alcanzaron aproximadamente 38,400 millones de toneladas de CO₂, impulsadas principalmente por el uso de carbón, gas natural y diversos procesos industriales [7].

Ante este escenario, la expansión de las fuentes de generación con bajas emisiones constituye uno de los principales objetivos de las políticas energéticas internacionales. En este sentido, la COP [8] establece como uno de sus objetivos incrementar de forma significativa la participación de las energías renovables y de la energía nuclear en el mix energético antes de 2030, al mismo tiempo que se continúa reduciendo la dependencia del carbón. Para alcanzar estas metas también será necesario modernizar las infraestructuras eléctricas mediante el aumento de la capacidad de transporte, almacenamiento y distribución de energía. Estas medidas permitirán integrar una mayor proporción de energías renovables, satisfacer el crecimiento de la demanda eléctrica y contribuir a una reducción progresiva de las emisiones globales de gases de efecto invernadero.

2.1.2. Eficiencia Energetica

2.2. Especificaciones Tecnicas del Entorno de Pruebas

En cuanto al entorno en el que se ha llevado a cabo el experimento, se ha utilizado un ordenador con una CPU Intel i7-4790K y 8 cores. Esta CPU es relativamente actual y nos

permite tomar medidas de casi cualquier parte del sistema utilizando RAPL [9]. También cuenta con una memoria RAM de 32 GB y 2 TB de memoria física HDD.

La tarjeta gráfica seleccionada es una NVIDIA Quadro M4000 [10], elegida entre el conjunto de GPUs disponibles en el laboratorio. En primer lugar, la tarjeta es compatible con la interfaz NVIDIA System Management (NVIDIA-SMI) [11], herramienta empleada para la monitorización y extracción de métricas relacionadas con el estado y el consumo energético de la tarjeta gráfica. Además, también ofrece soporte para las versiones de CUDA y NVIDIA Container Toolkit [12], lo cual permite el acceso a la GPU desde contenedores y la ejecución de cargas de trabajo con aceleración de hardware. Esta tarjeta gráfica incluye soporte para la tecnología NVENC, la cual permite usar los codificadores de hardware H.264 (AVC) y H.265 (HEVC) en diferentes formatos de codificación. En la Tabla 2.2 se muestran los formatos compatibles:

Codificador	Formato	CPU	GPU
H.264	YUV 4:2:0	x	X
H.264	YUV 4:4:4	X	X
H.264	Lossless	X	X*
H.265	4K YUV 4:2:0	X	X

Cuadro 2.1: Matriz de Capacidad de Codificación de Video de NVIDIA Quadro M4000

El SO sobre el que se han llevado a cabo las pruebas es un Debian 13.1-amd64 de 64 bits (x86_64) con la versión de kernel 6.12.90-2. Sobre este sistema se han instalado los drivers de NVIDIA con la versión 580.159.04 y la versión de CUDA 13.0. Se ha decidido instalar este sistema operativo debido principalmente a su estabilidad y su flexibilidad, habiendo redactado todo el proceso de instalación y preparación del entorno en el subapartado 3.1.3.

2.3. Selección de Herramientas de Consumo Energetico

Las herramientas encargadas de llevar a cabo la toma de métricas de consumo de las cargas de codificación desplegadas sobre contenedores de Docker tendrían que estar enfocadas a entornos de altas prestaciones compuestos por múltiples nodos. Además, debería ser capaz de tomar métricas de consumo de recursos hardware implicados, como la GPU, y poder diferenciar qué porcentaje de recursos ha sido utilizado por el contenedor.

Teniendo en cuenta estas necesidades, se ha realizado una búsqueda entre diversos repositorios, fuentes académicas y papers, como PAPER-KEPLER, y se ha llevado a cabo una selección de las herramientas que más se mencionan, tanto de forma directa como en la bibliografía. Estas dos herramientas han sido: Kepler [13] y Scaphandre [14].

Además, también se han proporcionado dos herramientas por parte de Juan Gutiérrez Aguado, uno de los directores del proyecto Eco-Stream: EnergyMeter, empleada como referencia de consumo energetico del sistema basado en las metricas del sistema obtenidas por RAPL; Y Monitor, una herramienta desarrollada por Juan Gutiérrez Aguado empleada como alternativa a las herramientas anteriores, la cual permite medir el consumo de una carga de trabajo desplegada sobre uno o varios contenedores de forma simultanea y muy precisa.

<i>HERRAMIENTA</i>	<i>NODO</i>	<i>PROCESO</i>	<i>CPU</i>	<i>GPU</i>
<i>Energy Meter</i>	X	-	X	-
<i>Kepler</i>	X	X	X	X
<i>Scaphandre</i>	X	X	X	-
<i>Monitor</i>	X	X	X	-

Cuadro 2.2: Capacidad de toma de metricas de las Herramientas

Kepler

Kubernetes Efficient Power Level Exporter (Kepler) es una herramienta de monitorización de consumo de energía pensada para ser utilizada en entornos con multiples nodos gestionados por Kubernetes sobre los cuales se pretende medir el consumo energetico de una carga de trabajo desplegada sobre un contenedor, y saber cual es el consumo energetico que este presenta.

Se basa en Intel RAPL para la obtencio de metricas de consumo del sistema y se apoya en nvidia-smi y sus librerias internar para obtener metricas de consumo de la GPU. Toma metricas cada 1 segundo, por defecto, y posteriormente aplica un post-procesamiento que resulta en un archivo compatible con Prometheus. Este es un software de monitorizacion de alertas compatible con Grafana, el cual no es necesario utilizar para tomar metricas con Kepler.

El conjunto de datos que proporciona Kepler en la salida se divide en tres bloques: El primero hace relacion al consumo energetico y metricas del programa, el segundo al consumo energetico a nivel de nodo y el tercero al consumo del proceso individual. Tanto en el segundo como el tercer bloque, distinguen metricas por zona de metricas(equivalentes a las que establece RAPL) y por core de la CPU.

Cabe destacar que pese a que Kepler es capaz de identificar que nodos han estado implicados en el proceso, no es capaz de diferenciar acerca del porcentaje de implicacion que cada nodo a tenido en el proceso, reparte el consumo energetico de forma equitativa.

Los resultado que se obtiene del posprocesamiento con kepler, se dividen en dos tipos: Enrgia (Energy), representada en microjulios de forma aculativa el consumo energetico del hardward; Y potencia (Power), representada en microwatios el consumo del hardware a lo largo del tiempo de medicion.

Segun se advierte en la documentacion oficial, Kepler presenta dificultades para medir de forma precisa el consumo energetico en entornos de carga mixta con patrones variables de cómputo y E/S, el modelo de atribución proporcional al tiempo de CPU puede ser inexacto, ya que no distingue entre el alto consumo de un proceso de cómputo intensivo y el menor gasto de uno que pasa gran parte del tiempo esperando operaciones de E/S, a pesar de que ambos puedan tener el mismo uso de CPU. Tambien presenta dificultades para realizar estimaciones sobre cargas de trabajo de alto rendimiento que usan instrucciones especializadas, como las vectoriales, la atribución basada únicamente en el tiempo de CPU no refleja el consumo energético real, que puede ser significativamente mayor. Por lo tanto, Kepler no es adecuado para decisiones de presupuesto energético absoluto ni para una optimización de grano fino que requiera mediciones precisas por proceso, ya que está diseñado para comparaciones relativas y análisis de tendencias, no para métricas exactas

a nivel de proceso individual.

Scaphandre

Scaphandre es un agente de monitorización especializado en la toma de métricas de consumo energético, diseñado por la empresa Hubblo para ayudar a organizaciones e individuos a cuantificar el consumo eléctrico de sus servicios tecnológicos, con un enfoque en la sostenibilidad. Su despliegue es versátil, pudiendo ejecutarse en múltiples entornos de múltiples formas. En sistemas GNU/Linux, se puede instalar desde los repositorios oficiales o mediante paquetes `*.deb*`, así como la posibilidad de desplegar un contenedor Docker que monitorice el consumo energético de un sistema y sus procesos. Además, Scaphandre puede compilarse con un conjunto limitado de características, lo que permite, por ejemplo, instalar únicamente el exportador para Prometheus Push Gateway o instalarse empleado Helm en entornos gestionada por Kubernetes.

La salida de la herramienta Scaphandre puede presentarse en formato `stdout` del sistema, JSON o según el esquema compatible con Prometheus. Dicho archivo contiene las métricas ofrecidas por RAPL en cada instante al iniciar la recolección, en base al valor de monitorización y la duración total de la métrica, los cuales son definidos al lanzar la toma de métricas con la herramienta. Este contiene las métricas del sistema en cada instante, pudiendo diferenciar entre consumo del sistema y consumo de un proceso. Cabe destacar que el conjunto de datos que se muestra representa el consumo energético en el instante de la toma de la métrica y que, por tanto, se repite n veces, dependiendo de cada cuánto se haya definido la toma de estos valores y la duración total de la monitorización, lo cual implica que se han de realizar las operaciones pertinentes a cada métrica en el postprocesado de los datos.

En cuanto a la toma de métricas, nos encontramos con algunas limitaciones; la herramienta emplea RAPL (Running Average Power Limit) para obtener datos de consumo energético, pero no puede acceder a `Nvidia_SMI` (System Management Interface) para medir el consumo de GPU. También presenta fuertes dependencias sobre el sistema operativo: requiere un entorno Linux y, en caso de utilizar procesadores AMD, una versión del kernel superior al 5.11. El uso de esta herramienta en nubes públicas se ve restringido porque obliga a modificar el hipervisor bajo el cual se lanzan las cargas de trabajo, las aplicaciones y las máquinas virtuales —una opción que la mayoría de los proveedores evita para evitar agotar su infraestructura al completo.

Energy Meter

Energy Meter es una de las herramientas desarrollada por el tutor de la Tesis Final de Master Juan Gutierrez Aguado, la cual tiene la capacidad de tomar métricas del consumo de todo el sistema empleando la librería `PyRAPL`. El formato de salida consiste en un registro JSON que contiene; el consumo energético del paquete (`pkg`), del apartado `*data*` y la suma total de los dos apartados anteriores.

Su principal valor radica en su bajo consumo, ya que opera únicamente consultando a RAPL al iniciarse la toma de métricas y al indicar su finalización. Además, se despliega fácilmente dentro de un contenedor y su uso requiere solo una solicitud HTTP. Sin embargo, aunque Energy Meter no permite tomar mediciones del consumo de la GPU ni distingue el consumo por proceso, es una herramienta ideal para servir como base comparativa frente

a herramientas como Kepler 2.3 y Scaphandre 2.3, en el contexto del consumo energetico total del sistema.

Monitor

Energy Meter es una de las herramientas proporcionadas por el tutor del Tesis Final de Master Juan Gutierrez Aguado; está diseñada como alternativa a las herramientas Kepler2.3 2.3 y Scaphandre, superando sus principales limitaciones. Permite adquirir métricas con granularidad muy baja, tanto del sistema completo como de procesos múltiples ejecutados en contenedores, y brinda información adicional sobre la carga de trabajo, su duración exacta y el procesamiento de datos. Además, ofrece la posibilidad de medir el consumo energético de la tarjeta gráfica mediante Nvidia SMI.

La herramienta se despliega mediante un archivo Docker Compose que incluye el contenedor de la aplicación y una base de datos MongoDB para almacenar los resultados de forma persistente, lo cual facilita su análisis posterior. Su despliegue es ligero y puede instalarse en cualquier entorno, independientemente del número de nodos, cumpliendo con las expectativas de las herramientas deseadas y resolviendo los puntos débiles de sus versiones anteriores.

Capítulo 3

Desarrollo de Proyecto

A continuacion se desarrollan los diferentes experimentos llevados a cabo, con la intencion de lograr cumplir los objetivos definidos en la fase de la introduccion [1](#).

3.1. Configuración de Entorno & Herramientas

3.1.1. Seleccion e Intalacion del Sistema Operativo

En un principio se decidió instalar el sistema operativo XUbuntu 26.04 en base a la recomendación de mi tutor de TFM, Juan Gutiérrez Aguado, ya que es un sistema operativo que él utiliza diariamente y sobre el que tiene una vasta experiencia de uso. Esto proporciona una capa extra de seguridad y de conocimiento sobre el entorno en el que se desarrollarían las pruebas, además del resto de recursos de conocimiento oficiales y no oficiales que existen sobre este sistema operativo.

Una vez finalizada la instalación, se hizo uso del gestor de versiones de drivers interno de Ubuntu, llamado `ubuntu-drivers`, para instalar la versión de los drivers de NVIDIA más compatible con la tarjeta gráfica NVIDIA Quadro M4000. Al comprobar si se podía acceder a los datos internos de consumo mediante `nvidia-smi`, se obtuvo un error que indicaba que no era capaz de comunicarse con la API interna de la tarjeta gráfica. Al revisar los mensajes proporcionados por el kernel, se pudo determinar que se trataba de un problema de compatibilidad de los drivers con la versión del kernel. Por este motivo se decidió instalar la versión 24.04 de XUbuntu [\[15\]](#), la cual hace uso de la versión 6.14 del kernel, solventando el problema de compatibilidad de los drivers.

Una vez reinstalado el sistema operativo, se procedió a reinstalar nuevamente los drivers de NVIDIA junto con los paquetes requeridos por `nvidia-smi`, de la misma forma que con la versión anterior. De nuevo, al tratar de acceder a los datos de consumo mediante `'nvidia-smi'`, apareció el mismo problema, pero al revisar los mensajes del kernel, se indicaba que el error no era de compatibilidad, sino la ausencia de un componente concreto llamado `'nvidia-drm'`. Este componente forma parte del conjunto de módulos del kernel de NVIDIA que permite al kernel comunicarse con la tarjeta gráfica de NVIDIA [\[16\]](#). Al tratar varias veces de reconstruir el módulo manualmente, sin lograr éxito, se optó por buscar información en foros oficiales de NVIDIA y foros de la comunidad Ubuntu y Xubuntu, llegando a la conclusión de que era un problema que sucedía en algunas versiones de XUbuntu debido a dos razones principales: La versión Nouveau de los drivers ,y la

incompleta compatibilidad de los drivers con XUbuntu.

El primer problema estaba relacionado con el controlador libre Nouveau, desarrollado por la comunidad mediante técnicas de ingeniería inversa. Aunque este controlador proporciona compatibilidad básica con un amplio número de tarjetas gráficas NVIDIA, no implementa completamente las funcionalidades propietarias ofrecidas por los controladores oficiales, entre ellas la interfaz NVML, utilizada por herramientas como nvidia-smi para acceder a métricas avanzadas de la GPU, incluido el consumo energético. Para solucionar este problema fue necesario deshabilitar la carga del módulo Nouveau y reinstalar exclusivamente los controladores oficiales proporcionados por NVIDIA.

El segundo problema estaba relacionado con el mecanismo de instalación automática de controladores proporcionado por la utilidad ubuntu-drivers. Tras consultar la documentación oficial y diferentes foros de las comunidades de Ubuntu y Xubuntu, se comprobó que esta herramienta puede instalar automáticamente la versión más reciente del controlador junto con paquetes pertenecientes a versiones diferentes o dejar dependencias de instalaciones anteriores. Esta situación puede provocar inconsistencias entre los módulos del kernel y las bibliotecas de usuario, impidiendo que componentes como nvidia-drm o nvidia-smi funcionen correctamente. Como consecuencia, fue necesario eliminar completamente todos los paquetes relacionados con NVIDIA y realizar una instalación manual de una versión del controlador compatible con la tarjeta gráfica y con el sistema operativo empleado.

Pese a todo ello, no se logró una comunicación adecuada con la API interna de la tarjeta gráfica, lo que impidió el acceso a las métricas de consumo energético mediante nvidia-smi. Es por ello que se optó por instalar otra distribución de Linux más flexible, con mayor compatibilidad con los drivers de NVIDIA. Finalmente, se decidió instalar Debian, ya que cumplía con los requerimientos necesarios, además de ser una distribución que a lo largo de los años personalmente había utilizado para diferentes proyectos; por tanto, tenía experiencia de uso y conocía su funcionamiento interno y su filosofía de trabajo.

Se seleccionó la versión Debian 13 (Trixie), que en el momento de realizar este trabajo se encontraba dos versiones por detrás de la versión estable más reciente (Debian 15 - Duke) [17], siguiendo las recomendaciones de la comunidad de Debian. Durante la instalación del sistema se habilitó la detección automática de la tarjeta gráfica y la instalación de los controladores propietarios de NVIDIA. Sin embargo, al intentar utilizar la GPU desde un contenedor Docker para ejecutar procesos de codificación acelerados mediante hardware, se obtuvo el error de compatibilidad mostrado en la Figura 3.1. Dicho error indicaba que la versión instalada del controlador no cumplía con la versión mínima requerida por el entorno de ejecución, siendo necesaria la versión 580 o superior. Tras actualizar los controladores a la versión requerida manualmente, fue posible acceder correctamente a la GPU desde los contenedores y ejecutar procesos de codificación mediante aceleración hardware, resolviendo así los problemas de compatibilidad entre los controladores de NVIDIA, Docker y las bibliotecas empleadas por FFMpeg.

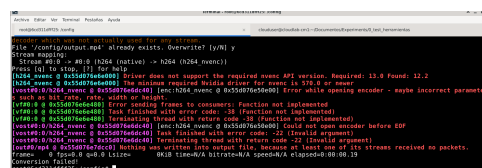


Figura 3.1: Error de compatibilidad con version de drivers de NVIDIA

3.1.2. Configuración e Instalación de Herramientas

A continuacio se exponen los procesos de instalacion, configuracion y modificaciones que se han llevado a cabo sobre las herramientas de toma de metricas. Todas esta herramientas monitorizan el consumo energetico del sistema y delas cargas de trabajo que desplegaran sobre contenedores basados en la imagen de ffmpeg gestionada por el grupo linux-server [18] [19].

Kepler

La instalación de Kepler en sistemas operativos Linux es relativamente sencilla, ya que la herramienta se distribuye en forma de ejecutable. Una vez descargado, dicho ejecutable debe añadirse a la variable de entorno *PATH* del sistema, permitiendo invocar la herramienta desde cualquier ubicación sin necesidad de indicar su ruta completa.

Una vez configurado el entorno, Kepler debe ejecutarse con privilegios de superusuario mediante el comando `sudo`, ya que necesita acceder a la interfaz *Running Average Power Limit* (RAPL) para recopilar las métricas de consumo energético, tal y como se muestra en el ejemplo de la Sección 3.1. La herramienta admite múltiples opciones de configuración, especificadas mediante *flags* durante su ejecución. Para la fase experimental desarrollada en este trabajo se han utilizado las siguientes:

- `-monitor.interval=1s`: establece un intervalo de muestreo de un segundo.
- `-metrics=container`: habilita la exportación de las métricas de consumo correspondientes a los contenedores monitorizados.
- `-metrics=node`: habilita la exportación de las métricas de consumo del nodo sobre el que se ejecuta la herramienta.
- `-experimental.gpu.enabled`: activa la recopilación de métricas asociadas a la GPU.

Por último, la salida estándar se redirige a `/dev/null`, ya que Kepler genera una gran cantidad de información relacionada con su configuración y estado de ejecución. Aunque esta información resulta útil durante las tareas de depuración, no es necesaria durante la ejecución de los experimentos y dificulta la interpretación de la salida por consola.

La obtención de las métricas se realiza mediante una petición HTTP utilizando la herramienta `curl` al puerto 28282, redirigiendo la respuesta a un archivo de salida para su posterior procesamiento. Una vez finalizada la recopilación de datos, Kepler debe detenerse manualmente, ya que, en caso contrario, continuará ejecutándose en segundo plano y seguirá recopilando métricas a la espera de nuevas peticiones, tal y como se muestra en la Sección 3.1.

Al tomar las primeras métricas con Kepler, se obtuvo un error en el apartado de consumo energético de la GPU, ya que marcaba como 0 el consumo energético cuando en realidad se había llevado a cabo una tarea de codificación. Al investigar los logs de salida de la aplicación, se observó un error al invocar una función interna llamada `'GetTotalEnergyConsumption'`, la cual pertenece a la librería `'libnvida-ml'` de `nvidia-smi`. Esta librería tiene la función de obtener el consumo de la tarjeta gráfica en cada instante que se hace

llamar a la función, y al parecer esta tenía problemas de compatibilidad con los drivers de NVIDIA que actualmente estaban instalados.

Tras investigar y comentar las posibles soluciones con mi tutor Juan, se decidió que se realizaría un hotfix sobre la herramienta Kepler, ya que se había logrado una estabilidad con los drivers y la reinstalación comprometería dicha estabilidad, además de no asegurar solventar el problema. Para ello se modificó aquellas funciones que llamaban a la función 'GetTotalEnergyConsumption' para que retornaran siempre 0 en su lugar. Una vez finalizado la modificación sobre el código de kepler, se recompiló el código sobre un contenedor de docker definido a partir de un Dockerfile 3.1.2.

Para comprobar que el hotfix ha funcionado adecuadamente, se lanzó una tarea de codificación y se monitorizó el consumo energético, tanto con kepler como con nvidia-smi. Tras analizar y procesar dando como resultado unos resultados muy cercanos, los cuales tenían una desviación entre sí del 1,07%. Es necesario remarcar, que kepler está devolviendo la potencia de la GPU durante el proceso de codificación, lo que implica que para obtener el consumo en Julious es necesario multiplicarlo por la duración del proceso de codificación.

```

1
2  #Toma de Metricas con Kepler
3  sudo ./kepler --metrics=container --monitor.interval=1s --metrics=node --experimental.
   gpu.enabled > /dev/null &)
4  wait_finish_container() #ESPERA FINALIZACION PROCESO DE CODIFICACION
5
6  #Recoleccion de metricas y finalizacion de la herramienta
7  curl -s http://localhost:28282/metrics > "nombre_archivo.metrics"
8  sudo pkill kepler 2>&1 &
9

```

Listing 3.1: Ejemplo de Ejecución y Recolección de métricas de la herramienta Kepler

```

1 # Build the binary
2 FROM golang:tip-trixie
3
4 # Install cross-compiler if building for a different architecture
5 RUN apt-get update && apt-get install -y --no-install-recommends \
6     gcc-x86-64-linux-gnu libc6-dev-amd64-cross && rm -rf /var/lib/apt/lists/*;
7
8 WORKDIR /workspace
9 COPY . .
10 RUN CROSS_CC=x86_64-linux-gnu-gcc; \
11     make build \
12     CGO_ENABLED=1 \
13     PRODUCTION=1 \
14     GOARCH=amd64 \
15     ${CROSS_CC:+CC=${CROSS_CC}} \

```

Scaphandre

Schaphandre puede ser desplegado de varias formas, pero la más sencilla es desplegarlo empleando el contenedor de Docker con la instrucción 3.1.2. Esta instrucción ha sido modificada para desplegar Schaphandre en un estado de 'sleep', a la espera de instrucciones, ya que únicamente queremos tomar métricas cuando el proceso de codificación inicie y no queremos que el arranque del contenedor añada ruido a las métricas de consumo.

```

1 docker run --name scaphandre -v /sys/class/powercap:/sys/class/powercap -v /proc:/proc
   --privileged -d --entrypoint "sleep" hubblo/scaphandre:1.0.2 3600
2

```

Para llevar a cabo la toma de métricas con Schaphandre, es necesario especificar cada cuánto tiempo se llevará a cabo la toma de métricas y durante cuánto tiempo, ya que, a diferencia de Kepler, Schaphandre sí que define un tiempo máximo de recolección de métricas. Además, también es necesario indicarle las métricas que se desean obtener y sobre qué archivo debe hacer el volcado de dichas métricas. Como se observa en el fragmento de código 3.1.2 correspondiente a la fase de experimentación 1 3.2.6, las órdenes de inicio de toma de métricas se llevan a cabo a través de la instrucción 'docker exec', indicando los parámetros de duración, tiempo de métricas y métricas que se desean obtener. En nuestro caso concreto, le indicamos que queremos obtener todas las métricas y que estas deben ser volcadas al archivo 'out.metrics'. Una vez finalizado el proceso de codificación, detenemos el proceso de forma forzosa y extraemos el archivo resultante del contenedor, para acto seguido apagarlo.

```

1  delta_t=1 # Tiempo de toma de metricas
2  t_metrics=60 #Tiempo de
3  # Start monitoring with scaphandre
4  docker exec -d scaphandre sh -c "scaphandre stdout -s ${delta_t} -t 60 --raw-metrics
   > out.metrics"
5
6  monitorizacion_contenedor() # Monitorizacion Proceso de transcodificacion en el
   contenedor
7  # Run process
8  # Stop scaphandre monitoring
9  docker exec scaphandre sh -c "pkill -f scaphandre"
10 scapfile=local_file_with_scaphandre_metrics.out
11 # Get data from scaphandre
12 docker cp scaphandre:/app/out.metrics ${scapfile}
13 # Change owner of file
14 sudo chown clouduser:clouduser ${scapFile}
15 # Stop scaphandre container
16 docker stop scaphandre
17

```

Al desplegar escaphandre de esta forma, tenemos control sobre el momento exacto del inicio de la recolección de toma de métricas y cuándo finaliza dicho proceso, eliminando posible ruido derivado del arranque del contenedor y los subprocesos asociados, logrando una replicabilidad en cada iteración para la fase experimental.

Energy Meter

Para instalar Energy Meter unicamente es necesario desplegar el docker compose creado por Juan Guatierrez 3.1.2, el cual se descarga del repositorio del proyecto la imagen energy-monitoring:1.0 y escuchapeticiones en el puerto 6000.

Para tomar metricas del sistema con la herramienta una vez docker compose esta arrancado, unicamente es necesario realizar una peticion curl de la siguiente forma `curl -s http://localhost:6000/api/start`. Y para recoger los datos de consumo, se debe realizar una nueva peticion `curl curl -s http://localhost:28282/metrics > "output.json"` pero esta vez redirigiendo la salida a un fichero, ya que al detener la toma de una métrica esta nos devuelve los datos de consumo del sistema en formato json.

```

1  services:
2    global-monitor:
3      image: energy-monitoring:1.0
4      container_name: global-energy-mon
5      privileged: true
6      ports:
7        - "6000:6000"
8      volumes:
9        - /var/run/docker.sock:/var/run/docker.sock
10       - /sys/fs/cgroup/system.slice:/sys/fs/cgroup/system.slice

```

```
11 - ./docker/app.py:/app/app.py
```

Monitor

Al igual que la anterior herramienta, Monitor ha sido desarrollada por Juan Gutierrez Aguado como alternativa a Kepler y Schaphandre. Para instalarla unicamente es necesario descargar el contenedor `*energy-monitoring:1.0*` del repositorio del proyecto de investigacion y desplegarlo utilizado docker compose 3.1.2. Este archivo compose expondra la herramienta en el puerto 5000 para recibir peticiones de monitorizacion.

El fragmento de código mostrado en la Sección 3.1.2 presenta un ejemplo del proceso de inicio y finalización de la monitorización, así como de la posterior obtención de las métricas recopiladas. Como puede observarse, tanto el inicio como la finalización de la monitorización se realizan mediante peticiones HTTP enviadas con la herramienta curl al puerto 5000, donde se encuentra desplegada la API de la herramienta.

Antes de iniciar el proceso de monitorización es necesario construir un payload en formato JSON, que será enviado en el cuerpo de la petición HTTP. Este payload contiene la información necesaria para identificar y configurar la carga de trabajo que se desea monitorizar.

En primer lugar, el atributo `workload_id` identifica de forma única el proceso de codificación, permitiendo asociar todas las métricas recopiladas a una misma carga de trabajo. A continuación, el atributo `container` especifica el nombre del contenedor cuya ejecución será monitorizada. Este parámetro resulta especialmente importante, ya que la herramienta permite monitorizar múltiples contenedores de forma simultánea.

Otro de los parámetros obligatorios es `gpu_indices`, mediante el cual se indica la GPU sobre la que se desea realizar la monitorización. Gracias a este mecanismo es posible trabajar con sistemas que disponen de varias tarjetas gráficas, asignando diferentes cargas de trabajo a cada una de ellas y obteniendo métricas independientes para cada contenedor y para cada GPU utilizada.

Finalmente, el atributo `persist_data` permite indicar si las métricas recopiladas deben almacenarse de forma persistente en la base de datos MongoDB. Este parámetro admite dos valores: 0, para deshabilitar el almacenamiento persistente, y 1, para guardar los datos una vez finalizada la monitorización.

La recoleccion de las metricas puede hacerse de dos formas diferentes; La primera es a traves de la respuesta que devuelve el comando que detiene la monitorizacion, el cual consisten en un json con el consumo energetico de varias partes del sistema asi como de la tarje grafica; Y a traves de una peticion a `*id_proceso/samples*`, la cual devuelve cada una de las metricas obtenida enla primera opcion pero enformato csv en el que cada linea corresponde a un instante de medicion.

Cabe destacar que, con el objetivo de garantizar la obtención de al menos diez muestras durante los procesos de transcodificación tanto en CPU como en GPU, fue necesario modificar el parámetro `*SAMPLES_PER_SECOND*` definido en el archivo Docker Compose de la herramienta, incrementando su valor de 1 a 3. De este modo, la herramienta pasa a realizar tres muestreos por segundo de monitorización, lo que permite obtener un número suficiente de muestras incluso en aquellos procesos de transcodificación cuya duración es

reducida

```

1  uid=$(uid)
2  payload='{"workload_id": "$uid",
3  "container": "$container",
4  "gpu_indices": [0],
5  "persist_data": 0}'
6  response=$(curl -s -H "Content-Type: application/json" -X POST -d "$payload" http://
7  localhost:5000/api/monitor)
8  id=$(echo "$response" | jq -r '.id')
9  echo "Monitoring container $id"
10
11 # Almacenamiento Resultados
12 output=$(curl -s -X DELETE http://localhost:5000/api/monitor?id=$id)
13 echo "Command: $workload_command" > ${infoFile}
14 docker exec $container sh -c "ls -al /config/output.mp4" >> ${infoFile}
15 echo "Output: $output" >> ${infoFile}
16 docker exec $container sh -c "ffprobe -v error -select_streams v -of default=
17 noprint_wrappers=1:nokey=1 -show_entries stream=r_frame_rate,width,height,duration,
18 bit_rate -of default=noprint_wrappers=1 /config/output.mp4" >> ${infoFile}
19
20 curl -s http://localhost:5000/api/monitor/workload/$uid/samples > ${csvFile}

```

```

1  services:
2    monitor:
3      image: routerdi1315.uv.es:33443/energy-monitoring:1.0
4      container_name: container-energy-mon
5      privileged: true
6      ports:
7        - "5000:5000"
8      volumes:
9        - /var/run/docker.sock:/var/run/docker.sock
10       - /sys/fs/cgroup/system.slice:/sys/fs/cgroup/system.slice
11       - ./data:/data
12       #- ./docker/app.py:/app/app.py
13       #- ./docker/monitor.py:/app/monitor.py
14       #- ./docker/mdb.py:/app/mdb.py
15     environment:
16       - MONGO_HOST=mongodb
17       - MONGO_PORT=27017
18       - NVIDIA_VISIBLE_DEVICES=all
19       - MONITOR_GPU=True
20       - SAMPLES_PER_SECOND=3
21     runtime: nvidia
22
23  mongodb:
24    image: mongo:8.2.2-noble
25    container_name: mongodb
26    volumes:
27      - ./mongodb/data:/data/db
28    ports:
29      - "27017:27017"
30    command:
31      - "--quiet"

```

3.1.3. Estado Final del Sistema

Una vez finalizado el conjunto de ajustes y de herramientas necesarias para llevar a cabo las diferentes fases experimentales, se han tomado métricas de consumo del sistema. El consumo energético del sistema, teniendo arrancada la herramienta EM y el contenedor de codificación (en un estado de sleep), se obtiene que el sistema está consumiendo de forma pasiva unos 544-545 julios. Por otra parte, la tarjeta gráfica está consumiendo unos 11 vatios de forma pasiva.

Sabiendo cuál es el estado de consumo del sistema en reposo, es posible empezar con las fases de experimentación que se han llevado a cabo a lo largo de este proyecto de

investigación.

3.2. Selección de Herramienta para Métricas de Consumo

3.2.1. Muestra de Videos Para la fase de Experimentacion 1

3.2.2. Metricas de consumo: Transcodificacion usando 1 contenedor sobre CPU+GPU

3.2.3. Metricas de consumo: Transcodificacion usando 1 contenedor sobre CPU

3.2.4. Metricas de consumo: Transcodificacion usando 1 contenedor con 3 cores de CPU limitados

3.2.5. Metricas de consumo: Transcodificacion usando 2 contenedores simultaneos con limite de 3 cores cada uno

3.2.6. Conclusiones fase 1 de la experimentacion

3.3. Fase de Experimentacion 2: Consumo energetico en relacion al SITI y el Numero de Threas

3.3.1. Muestra de videos para la fase 2 de experimentacion

El conjunto de videos que será utilizado para en el apartado del desarrollo [3.3](#) proviene de la base de videos que el proyecto EcoStrema tiene, y que proviene principalmente de [\[20\]](#). Estos videos tiene la caracteristica que estan codificados en formato H.265 sin perdida de calidad y sin compresion, ademas de

3.3.2. Seleccion de muestras Basado en SI y TI

Para la selección del conjunto de videos que serán utilizados durante la fase de experimentación 2 [3.3](#), se espera poder comprobar si el contenido del video, y más concretamente, la cantidad de información que contiene, afecta a la cantidad de energía que consume el contenedor durante el proceso de codificación. Es por este motivo que se han empleado los parámetros Spatial Information (SI) y Temporal Information (TI) para la selección de chunks que compondrán los videos.

Estos parámetros de calidad subjetiva se encuentran definidos por el International Telecommunication Union Telecommunication Standardization Sector (ITU) [\[21\]](#), y se emplean de forma conjunta como métricas de calidad comprendida entre 0 y 100, donde Spatial Information (SI) representa la cantidad de detalles y objetos que aparecen en una

escena, y el Temporal Information (TI) define la cantidad de cambios que suceden en una escena a lo largo del tiempo.

Para ellos, se han seleccionado los vídeos mostrados en la Tabla 3.3.2. A partir de cada uno de ellos se calcularán los índices de Información Espacial (SI) e Información Temporal (TI) de sus correspondientes chunks. Posteriormente, se generarán vídeos de 10 segundos de duración a partir de un único chunk, el cual se reproducirá de forma cíclica hasta completar el tiempo establecido. Este procedimiento permite mantener constantes los valores de SI y TI durante toda la secuencia, evitando que las variaciones del contenido afecten al análisis del consumo energético.

Video	Origen
Eldorado	Cable Labs
Skateboarding	Cable Labs
LifeUntouched	Cable Labs
LiftingOff	Cable Labs
Sintel	Bender
SecondsThatCount	Cable Labs
IndoorSoccer	Cable Labs

Cuadro 3.1: Videos Seleccionados para las pruebas

Con el objetivo de disponer de una muestra representativa de diferentes tipos de contenido, los chunks se han clasificado en cuatro categorías según sus valores de SI y TI, tal y como se muestra en la Lista 3.3.2. Para cada categoría se seleccionarán cinco chunks, obteniendo un total de veinte vídeos para la experimentación. Esta muestra permitirá analizar la posible relación entre las características espaciales y temporales del contenido y el consumo energético asociado al proceso de codificación.

- Alto TI y bajo SI (High TI – Low SI).
- Bajo TI y alto SI (Low TI – High SI).
- Bajo TI y bajo SI (Low TI – Low SI).
- Alto TI y alto SI (High TI – High SI).

Para lograr calcular y visualizar los valores de SI y TI correspondientes a cada chunk, se han desarrollado dos scripts en Python. El primero A.1.1 emplea la biblioteca *siti-tools* [22], disponible en el gestor de paquetes de Python PyPI, para calcular las métricas de Información Espacial (SI) e Información Temporal (TI) de cada chunk y almacenar los resultados en un archivo CSV.

El segundo script A.1.1 utiliza dicho archivo como entrada para generar una gráfica de dispersión, en la que el eje X representa los valores de SI y el eje Y los valores de TI. Cada punto de la gráfica corresponde a un chunk, cuya posición viene determinada por las métricas calculadas previamente.

3.3.3. Consumo energetico en relacion al SITI

3.3.4. Consumo energetico al limitar threads y numero de cores

Capítulo 4

Pruebas y Resultados

Capítulo 5

Conclusiones y Trabajo Futuro

Apéndice A

Apéndice

A.1. Subapendice: Experimentacion

A.1.1. Experimetacion Fase 2

Codigo Python para Calculo y Visualizacio de metricas SITI

CODIGO PYTHON SITI

CODIGO PYTHON PLOTTER

Bibliografía

- [1] Rupinder Kaur, Gurjinder Kaur, and Major Singh Goraya. A workflow scheduling algorithm with task classification for reducing energy consumption and makespan in cloud. *Sustainable Computing: Informatics and Systems*, 50:101323, June 2026.
- [2] Samira Afzal, Narges Mehran, Zoha Azimi Ourimi, Farzad Tashtarian, Hadi Amirpour, Radu Prodan, and Christian Timmerer. A Survey on Energy Consumption and Environmental Impact of Video Streaming, January 2024. arXiv:2401.09854 [cs.MM].
- [3] Kashif Nizam Khan, Mikael Hirki, Tapio Niemi, Jukka K. Nurminen, and Zhonghong Ou. RAPL in Action: Experiences in Using RAPL for Power Measurements. *ACM Trans. Model. Perform. Eval. Comput. Syst.*, 3(2):9:1–9:26, March 2018.
- [4] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D. Joseph, Randy Katz, Andy Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, Ion Stoica, and Matei Zaharia. A view of cloud computing. *Commun. ACM*, 53(4):50–58, April 2010.
- [5] Ashkan Safari, Hoda Sorouri, Afshin Rahimi, and Arman Oshnoei. A systematic review of energy efficiency metrics for optimizing cloud data center operations and management. *Electronics*, 14(11):2214, May 2025.
- [6] Electricity 2026 – analysis.
- [7] Global energy review 2026 – analysis.
- [8] Share:. Cop2030 presidency releases executive report and outlines next steps to accelerate global climate implementation.
- [9] Intel core i7-4790k processor (8m cache, up to 4.40 ghz) - product specifications.
- [10] nvidia.com. https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/documents/75509_DS_NV_Quadro_M4000_US_NV_HR.pdf. [Accessed 05-07-2026].
- [11] nvidia-smi-documentation.
- [12] Installing the nvidia container toolkit — nvidia container toolkit.
- [13] Kepler.
- [14] Introduction - scaphandre documentation.
- [15] Xubuntu – Releases.
- [16] Kernel modules — nvidia nriver installation guide.
- [17] Debian – versiones de debian.

-
- [18] ffmpeg - linuxserver.io.
 - [19] Home - linuxserver.io.
 - [20] 4k Video.
 - [21] P.910:subjective video quality assessment methods for multimedia applications.
 - [22] siti-tools: Functions to calculate spatial information / temporal information according to itu-tp.910.